



NED University of Engineering and Technology

Department of Computer Science and IT

SSCS (Specialization in Data Science)

CT-361 Artificial Intelligence and Expert System

CCP - Project Report

Urban Traffic Monitoring System

Group members:

Neha Laeeq	DT-22002
Hassan Hayat	DT-22010
Shayan	DT-22037
Ahmed Hassan	DT-22041

Index:

Table of Contents

Index:	2
1. System Overview:	3
2. Modules	3
2.1. Image and Video Detection:	3
2.1.1. Objective:	3
2.1.2. Haar Technique:	4
2.1.3. How Haar Cascade Works:	4
2.1.4. Explanation of Code:	4
2.1.5. Results:	8
2.1.7. How YOLO Works:	9
2.1.8. Explanation of Code:	9
2.2. Sound Analysis:	14
2.2.1. Objective	14
2.2.2. How Sound Analysis/Speech Recognition Works:	14
2.2.3. Explanation of Code:	15
2.3. Graph Traversal Algorithms:	19
2.3.1. Objective	19
2.3.2. Breadth-First Search (BFS):	19
2.3.3. How BFS Works:	19
2.3.4. Depth-First Search (DFS):	20
2.3.5. How DFS Works:	20
2.3.6. Explanation of Code:	20

1. *System Overview:*

The **Intelligent Urban Traffic Monitoring System** is a multi-module framework designed to autonomously monitor and analyze urban traffic. It operates in real-time, processing data from diverse sources, including cameras and audio sensors deployed across critical intersections and road segments.

The system consists of three main modules:

1. **Image and Video Detection Module:** Detects and classifies different types of vehicles using AI-based object detection algorithms like Haar-Feature and YOLO.
2. **Sound Analysis Module:** Identifies unusual urban sounds, such as sirens or crashes, using sound classification techniques to detect and prioritize incidents.
3. **Graph-Based Traffic Flow Module:** Models the road network as a graph, utilizing traversal algorithms like BFS and DFS to suggest optimal routes and reduce congestion.

Together, these modules create a comprehensive system that integrates visual, auditory, and graph-based data to monitor and optimize urban traffic flow effectively.

2. *Modules*

2.1. *Image and Video Detection:*

2.1.1. *Objective:*

To detect and classify vehicles (e.g., cars) in real-time using image and video data. The module employs two methods:

- **Haar Cascade** for basic object detection.
- **YOLO (You Only Look Once)** for advanced and accurate vehicle detection.

2.1.2. *Haar Technique:*

Haar Cascade is a machine learning-based algorithm designed for object detection in images or videos. It utilizes pre-trained classifiers to identify features such as edges, corners, and lines, allowing it to recognize specific objects. The algorithm systematically scans the image by sliding a window over it and applies the classifiers to detect the target object. Renowned for its efficiency and accuracy, Haar Cascade is commonly used in real-time applications, including facial recognition in cameras and video surveillance systems.

2.1.3. *How Haar Cascade Works:*

- **Classification Process:** Distinguishes between positive data points (parts of the object being detected) and negative ones (areas without the object).
- **Cascaded Filters:** Applies a sequence of filters, each designed to detect specific features within an image.
- **Efficiency:** Quickly eliminates non-matching regions, making it suitable for real-time tasks.
- **Resource-Friendly:** Requires minimal computational power and is straightforward to implement.

2.1.4. *Explanation of Code:*

```
input_video = '../inputs/road.mp4'  
haar_output = '../outputs/haar_video.mp4'
```

- **input_video:** Path to the input video file (`road.mp4`) that will be processed.
- **haar_output:** Path where the output video file (`haar_video.mp4`) with detected cars will be saved.

```
car_detector = cv2.CascadeClassifier('cars.xml')
```

Purpose: Initializes a Haar Cascade classifier using a pre-trained XML file (`cars.xml`). This file contains the data required to detect cars in images or videos.

```
if car_detector.empty():  
    print("Error: Failed to load cascade file. Please verify the file path.")  
    exit()
```

Purpose: Ensures the `cars.xml` file is loaded correctly. If the file is not found or is corrupted, the program will terminate with an error message.

```
video_capture = cv2.VideoCapture(input_video)
```

Purpose: Loads the input video for frame-by-frame processing.

- **cv2.VideoCapture:** A function in OpenCV used to read videos from a file or camera.

```
if not video_capture.isOpened():  
    print("Error: Unable to open input video.")  
    exit()
```

Purpose: Verifies that the input video file is accessible. If not, the program terminates.

```
frame_rate = int(video_capture.get(cv2.CAP_PROP_FPS))  
frame_width = int(video_capture.get(cv2.CAP_PROP_FRAME_WIDTH))  
frame_height = int(video_capture.get(cv2.CAP_PROP_FRAME_HEIGHT))  
codec = cv2.VideoWriter_fourcc(*'mp4v')  
haar_writer = cv2.VideoWriter(haar_output, codec, frame_rate, (frame_width, frame_height))
```

Purpose: Extracts properties of the input video for use in the output video.

- **cv2.CAP_PROP_FPS:** Retrieves the video's frame rate.
- **cv2.CAP_PROP_FRAME_WIDTH** and **cv2.CAP_PROP_FRAME_HEIGHT:** Retrieves the width and height of each frame.
- **cv2.VideoWriter_fourcc:** Specifies the codec (e.g., `mp4v`) for the output video.

- **cv2.VideoWriter**: Initializes the writer object to save the processed video.

```
while True:
    success, video_frame = video_capture.read()
    if not success:
        print("Video processing completed.")
        break
```

Purpose: Reads the video frame by frame. If no more frames are available, the loop terminates.

- **cv2.VideoCapture.read** – Retrieves the next frame from the video.

```
gray_frame = cv2.cvtColor(video_frame, cv2.COLOR_BGR2GRAY)
```

Purpose: Converts the color video frame to grayscale. Haar Cascade works best with grayscale images as it simplifies the data for pattern recognition.

Grayscale Conversion: Reduces the computational complexity by eliminating color information.

```
detected_cars = car_detector.detectMultiScale(
    gray_frame,
    scaleFactor=1.2,
    minNeighbors=4,
    minSize=(40, 40)
)
```

Purpose: Detects cars in the grayscale frame using the Haar Cascade classifier.

Terms:

- **scaleFactor**: Specifies how much the image size is reduced at each scale. Higher values make detection faster but less accurate.
- **minNeighbors**: The number of neighbors each rectangle should have to retain it. Higher values reduce false positives.
- **minSize**: Minimum size of the object to be detected (in pixels).

```

for (x, y, w, h) in detected_cars:
    cv2.rectangle(video_frame, (x, y), (x + w, y + h), (255, 0, 0), 2)
    cv2.putText(
        video_frame,
        "Car",
        (x, y - 15),
        cv2.FONT_HERSHEY_SIMPLEX,
        0.5,
        (255, 0, 0),
        2
    )

```

Purpose: For each detected car, draw a blue bounding box around it and label it as "Car".

- **cv2.rectangle:** Draws a rectangle to visualize the detection.
- **cv2.putText:** Adds the "Car" label above each bounding box.

```

haar_writer.write(video_frame)

```

Purpose: Saves the annotated frame to the output video.

- **cv2.VideoWriter.write** – Appends processed frames to the output video file.

```

video_capture.release()
haar_writer.release()

```

Purpose: Frees up system resources by closing the video capture and writer objects.

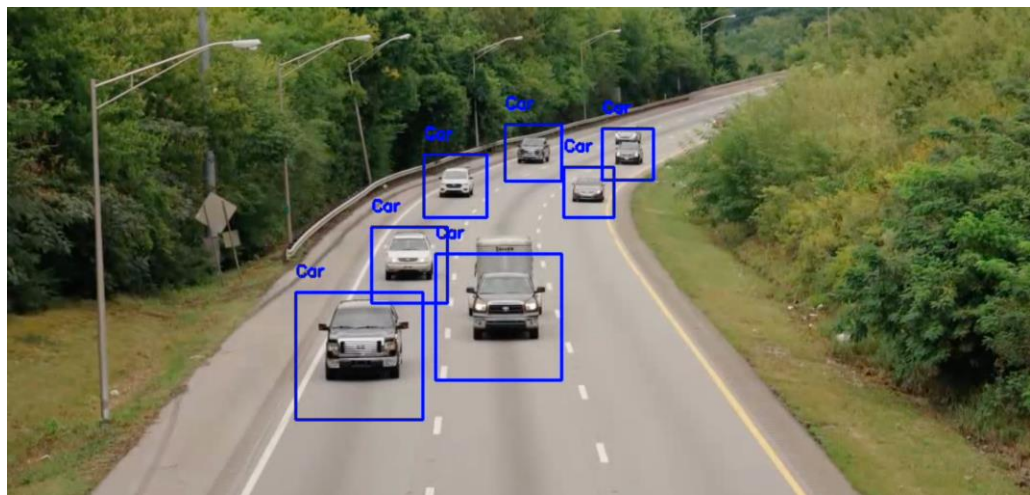
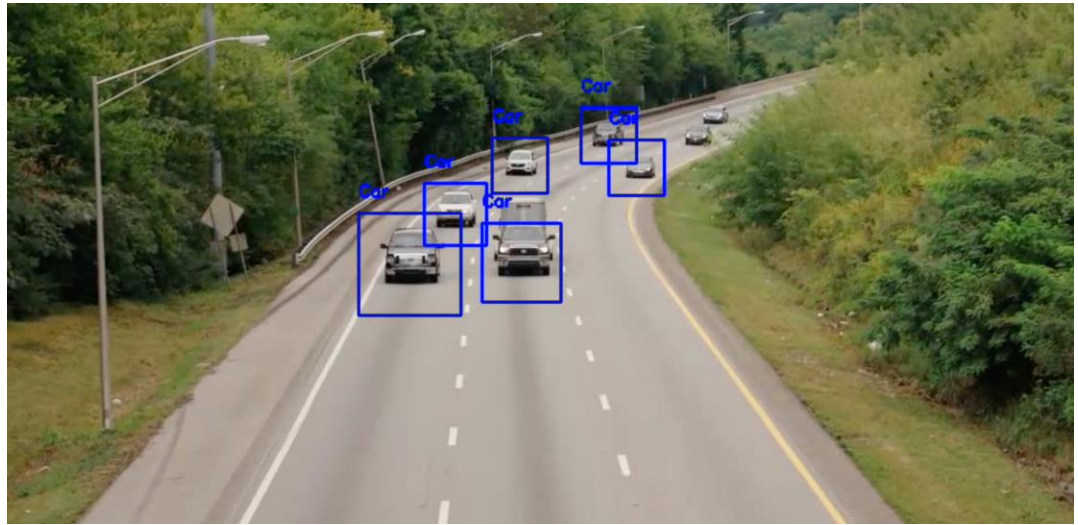
```

print(f"Video processed using Haar Cascade and saved as {haar_output}")

```

Purpose: Prints a confirmation message with the output file path.

2.1.5. Results:



2.1.6. *YOLO technique:*

YOLO is a real-time object detection algorithm known for its speed and accuracy. It treats object detection as a single regression problem, predicting bounding boxes and class probabilities directly from an image. It's widely used in areas like autonomous vehicles, security surveillance, and real-time object tracking.

2.1.7. *How YOLO Works:*

- **Single Neural Network Architecture:** YOLO uses a single convolutional neural network (CNN) to process the entire image in one pass, making it efficient.
- **Grid-Based Detection:** The image is divided into grids, with each grid cell responsible for detecting objects in its region.
- **Bounding Box Prediction:** YOLO predicts the position, size, and confidence score of each object in the form of bounding boxes.
- **Real-Time Performance:** Its architecture ensures high-speed detection, ideal for time-sensitive applications.
- **Feature Extraction:** It captures both spatial and contextual information from the image, leading to fewer false detections.

2.1.8. *Explanation of Code:*

```
source_video = '../inputs/road.mp4'  
yolo_output = '../outputs/yolo_video.mp4'
```

- **source_video:** Path to the input video file (`road.mp4`) that will be processed using YOLO for object detection.

- **yolo_output**: Path where the processed video with YOLO-based object detection will be saved (**yolo_video.mp4**).

```
object_detector = YOLO('yolov8x.pt')
```

Purpose: This line initializes the YOLOv8 object detector using a pre-trained weights file (**yolov8x.pt**), which contains the deep learning model for detecting various objects in images and videos.

```
video_reader = cv2.VideoCapture(source_video)
```

Purpose: Opens the source video (**road.mp4**) for reading frames. The **cv2.VideoCapture** function from OpenCV allows frame-by-frame video processing.

```
if not video_reader.isOpened():  
    print("Error: Could not load source video.")  
    exit()
```

Purpose: Checks if the video file was successfully opened. If not, it prints an error message and stops the program.

```
video_fps = int(video_reader.get(cv2.CAP_PROP_FPS))  
video_width = int(video_reader.get(cv2.CAP_PROP_FRAME_WIDTH))  
video_height = int(video_reader.get(cv2.CAP_PROP_FRAME_HEIGHT))
```

Purpose: These lines extract important video properties such as frames per second (**video_fps**), width (**video_width**), and height (**video_height**).

```
video_codec = cv2.VideoWriter_fourcc(*'mp4v')  
yolo_writer = cv2.VideoWriter(yolo_output, video_codec, video_fps, (video_width, video_height))
```

Purpose: Initializes a video writer object that will be used to save the processed video with detected objects. The codec (**'mp4v'**) is used for writing **.mp4** files.

```
while True:
    success, current_frame = video_reader.read()
    if not success:
        print("YOLO detection completed.")
        break
```

Purpose: Continuously reads frames from the video. If reading fails (e.g., the video ends), the loop breaks, and the program stops processing.

```
yolo_results = object_detector(current_frame)
```

Purpose: Runs the YOLO model on the current video frame to detect objects. The result contains bounding boxes, class IDs, and confidence scores for detected objects

```
for detection in yolo_results[0].boxes:
    x_start, y_start, x_end, y_end = detection.xyxy[0].numpy()
    confidence = detection.conf[0].item()
    class_id = int(detection.cls[0].item())
    detected_label = object_detector.names[class_id]
```

Purpose: This loop iterates through each detected object in the frame, extracting the bounding box coordinates (`x_start`, `y_start`, `x_end`, `y_end`), confidence score, and class ID (e.g., car, pedestrian).

```
cv2.rectangle(current_frame, (int(x_start), int(y_start)), (int(x_end), int(y_end)), (0, 0, 255), 2)
cv2.putText(
    current_frame,
    f'{detected_label} {confidence:.2f}',
    (int(x_start), int(y_start) - 15),
    cv2.FONT_HERSHEY_SIMPLEX,
    0.5,
    (0, 0, 255),
    2
)
```

Purpose: Draws a red rectangle around the detected object using the bounding box coordinates. The label (e.g., "car") and confidence score (e.g., 0.85) are displayed above the box.

```
yolo_writer.write(current_frame)
```

Purpose: Writes the current frame (with detections) to the output video file.

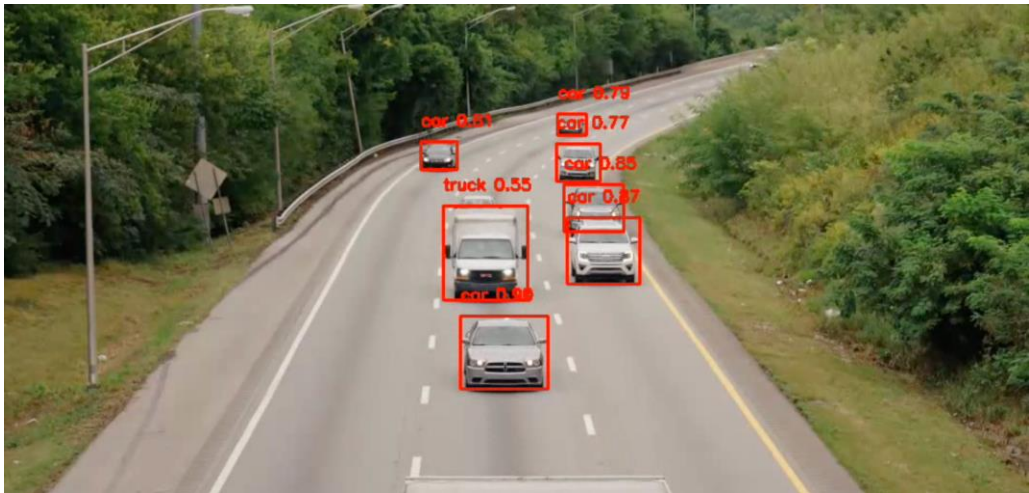
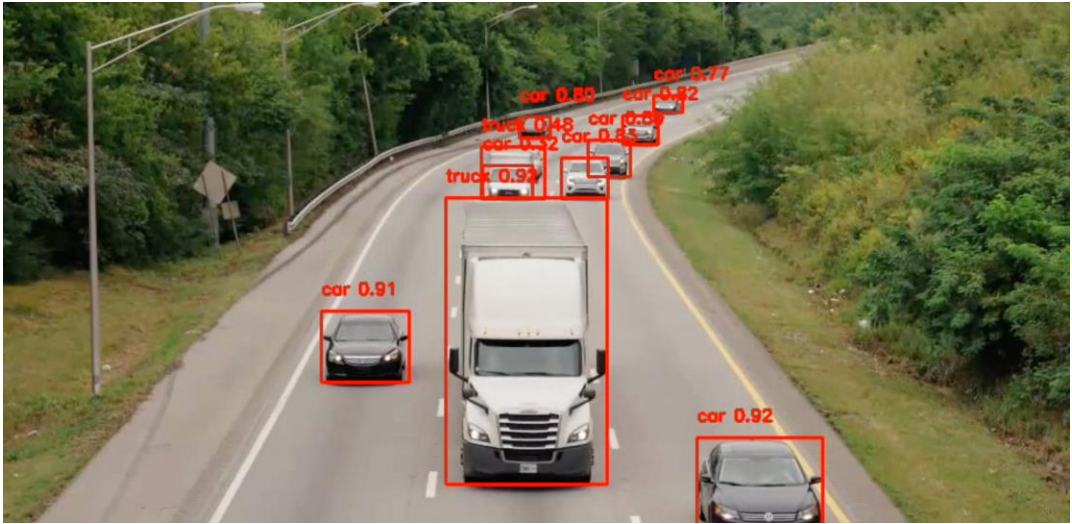
```
video_reader.release()  
yolo_writer.release()
```

Purpose: Releases the video reader and writer objects to free up system resources.

```
print(f"YOLO-based processed video saved as {yolo_output}")
```

Purpose: Displays a message indicating the output video file has been successfully saved.

2.1.9. *Results:*



2.2. *Sound Analysis:*

2.2.1. *Objective*

Speech Recognition and Sound Analysis involves the interpretation of audio signals to extract meaningful information or convert spoken words into text. These systems leverage signal processing and machine learning techniques to analyze and understand audio data. Common applications include virtual assistants, transcription services, and real-time translation tools.

2.2.2. *How Sound Analysis/Speech Recognition Works:*

- **Feature Extraction:**
 - Converts raw audio signals into distinctive features such as Mel Frequency Cepstral Coefficients (MFCCs), spectrograms, or chromagrams.
 - Breaks down audio into smaller frames for detailed analysis of sound patterns.
- **Acoustic Modeling:**
 - Establishes a relationship between audio features and linguistic units (e.g., phonemes) using models like Hidden Markov Models (HMMs) or Deep Neural Networks (DNNs).
- **Language Modeling:**
 - Transforms recognized phonemes into coherent words and sentences using Natural Language Processing (NLP) techniques to ensure correct grammar, syntax, and word probabilities.
- **Decoding:**
 - Combines both the acoustic and language models to produce the final transcription of the spoken audio.
- **Real-Time Processing:**

- Systems such as Google Speech-to-Text, Apple Siri, and Amazon Alexa perform real-time audio processing, enabling quick responses and transcription of spoken language.

2.2.3. *Explanation of Code:*

```
import tensorflow as tf
import tensorflow_hub as hub
import librosa
```

Purpose: Imports essential libraries.

- **TensorFlow:** A machine learning framework used for loading and processing the pre-trained audio model.
- **TensorFlow Hub (hub):** Allows loading a pre-trained model (YAMNet in this case) from Google's repository.
- **Librosa:** A Python library for analyzing audio signals, used here to load and process the audio file.

```
audio_model = hub.load('https://tfhub.dev/google/yamnet/1')
```

Purpose: Loads the pre-trained **YAMNet** model, which classifies environmental sounds, including urban sounds such as sirens, honks, and crashes.

Term: **YAMNet** is a deep neural network trained on AudioSet to recognize over 500 sound classes, including many relevant to urban traffic.

```
label_map_path = audio_model.class_map_path().numpy().decode('utf-8')
sound_labels = [
    line.strip().split(',')[2]
    for line in tf.io.gfile.GFile(label_map_path)
    if ',' in line and not line.startswith('index')
]
```

Purpose: Loads and parses the class labels used by YAMNet. These labels

represent different types of sounds (e.g., sirens, honking, crashes) that the model can recognize.

```
city_sound_categories = {
    "blockage_honk": ["Toot"],
    "emergency_siren": ["Siren", "Police car (siren)", "Ambulance (siren)"],
    "crash": ["Breaking"],
}

detection_thresholds = {
    "blockage_honk": 0.12,
    "emergency_siren": 0.15,
    "crash": 0.10,
}
```

Purpose: Defines categories of sounds relevant to the urban environment (e.g., honking for blockages, sirens for emergencies) and their respective detection thresholds.

```
def analyze_audio_clip(audio_clip_path):
    audio_waveform, sample_rate = librosa.load(audio_clip_path, sr=16000)
    audio_waveform = tf.convert_to_tensor(audio_waveform, dtype=tf.float32)

    probabilities, embeddings, spectrogram = audio_model(audio_waveform)
```

Purpose: Loads the audio file, converts it to the required format (16 kHz sample rate), and runs it through the YAMNet model to obtain predictions (probabilities for each sound class).

```
print("\nDebugging Frame Probabilities:")
for frame_index, frame_scores in enumerate(probabilities.numpy()):
    print(f"Frame {frame_index}:")
    for label_idx, probability in enumerate(frame_scores):
        if probability > 0.1:
            print(f" - {sound_labels[label_idx]}: {probability:.4f}")
```

Purpose: Prints the sound labels with probabilities higher than 0.1 for each frame, which helps in debugging and understanding which sounds the model is detecting and with what confidence.

```

def identify_city_sounds(probabilities):
    detected_events = {"alerts": [], "details": []}
    frame_probabilities = probabilities.numpy()

    for frame_index, frame_scores in enumerate(frame_probabilities):
        for category, sound_keywords in city_sound_categories.items():
            for keyword in sound_keywords:
                try:
                    label_idx = sound_labels.index(keyword)
                except ValueError:
                    print(f"Keyword '{keyword}' not found in sound_labels.")
                    continue
                if frame_scores[label_idx] >= detection_thresholds[category]:
                    if category not in detected_events["alerts"]:
                        detected_events["alerts"].append(category)
                    detected_events["details"].append({
                        "sound": keyword,
                        "confidence": frame_scores[label_idx],
                        "frame_idx": frame_index,
                    })
    return detected_events

```

Purpose: Checks the predicted probabilities for each frame and compares them with pre-defined detection thresholds. If the probability of a specific sound exceeds the threshold, it flags the sound event.

```

if sound_events["alerts"]:
    print("\n*** ALERT: CITY SOUND EVENTS DETECTED ***")
    for alert in sound_events["alerts"]:
        print(f"- Sound Category: {alert}")
    for detail in sound_events["details"]:
        print(
            f" - Detected Sound: {detail['sound']}, "
            f"Confidence: {detail['confidence']:.2f}, Frame: {detail['frame_idx']}"
        )
else:
    print("\nNo significant city-related sounds detected.")

```

Purpose: Prints a summary of any detected city-related sound events, including the type of sound (e.g., siren, honking) and the confidence score. If no significant sounds are detected, it reports that as well.

```
audio_path = "../inputs/police-siren.mp3"  
analyze_audio_clip(audio_path)
```

Purpose: This final section specifies the path to the audio clip (in this case, a police siren) and runs the analysis on it.

2.2.4. *Results:*

```
** ALERT: CITY SOUND EVENTS DETECTED **  
· Sound Category: emergency_siren  
  - Detected Sound: Siren, Confidence: 0.56, Frame: 0  
  - Detected Sound: Police car (siren), Confidence: 0.56, Frame: 0  
  - Detected Sound: Siren, Confidence: 0.47, Frame: 1  
  - Detected Sound: Police car (siren), Confidence: 0.41, Frame: 1  
  - Detected Sound: Siren, Confidence: 0.59, Frame: 2  
  - Detected Sound: Police car (siren), Confidence: 0.38, Frame: 2  
  - Detected Sound: Siren, Confidence: 0.25, Frame: 3  
  - Detected Sound: Police car (siren), Confidence: 0.36, Frame: 3  
  - Detected Sound: Siren, Confidence: 0.16, Frame: 4  
  - Detected Sound: Police car (siren), Confidence: 0.22, Frame: 4  
  - Detected Sound: Siren, Confidence: 0.20, Frame: 5  
  - Detected Sound: Police car (siren), Confidence: 0.31, Frame: 5  
  - Detected Sound: Siren, Confidence: 0.46, Frame: 6  
  - Detected Sound: Police car (siren), Confidence: 0.18, Frame: 6
```

2.3. Graph Traversal Algorithms:

2.3.1. Objective

Graph traversal refers to the process of visiting all the nodes in a graph in a systematic way. It is fundamental in graph theory and is used in various applications, such as searching for specific nodes, finding the shortest path, and detecting cycles. In the context of a city's road network, graph traversal algorithms can help identify optimal routes, detect traffic congestion, and model the movement of vehicles through intersections.

2.3.2. Breadth-First Search (BFS):

BFS is an algorithm that explores all the nodes at the present depth level before moving on to nodes at the next depth level. It is typically implemented using a queue to maintain the nodes to be explored.

2.3.3. How BFS Works:

- Start from a selected node (source).
- Visit all unvisited neighboring nodes and add them to the queue.
- Pop the next node from the queue and repeat the process until all nodes are explored or a specific goal node is found.
- BFS is optimal for finding the shortest path in an unweighted graph because it explores all nodes level by level.

2.3.4. *Depth-First Search (DFS):*

DFS is an algorithm that explores as far as possible along each branch before backtracking. It uses a stack to keep track of the nodes to be visited next.

2.3.5. *How DFS Works:*

- Start from a selected node (source).
- Visit an unvisited neighboring node and push it onto the stack.
- Continue to explore deeper along the branch until there are no more unvisited neighbors.
- Backtrack to the previous node and continue the process until all reachable nodes are visited.
- DFS can be used to explore all possible routes, even if the goal is not to find the shortest path.

2.3.6. *Explanation of Code:*

```
import networkx as nx
import matplotlib.pyplot as plt
city_graph = nx.DiGraph()
```

Purpose: We are importing the necessary libraries. `networkx` is used to create and manipulate graphs, while `matplotlib` is used for visualizing the graph.

Explanation: `nx.DiGraph()` initializes a directed graph object that represents the city's road network. The graph will have intersections as nodes and roads as edges.

```

coordinates = {
    "CliftonBeach": (0, 10),
    "MazarQuaid": (1, 9),
    "DolmenMall": (2, 8),
    "EmpressMarket": (3, 7),
    "PortGrand": (4, 6),
    "FrereHall": (5, 5),
    "KarachiZoo": (6, 4),
    "HawksBay": (7, 3),
    "TariqRoad": (8, 2),
    "LuckyOneMall": (9, 1),
    "Saddar": (3, 9),
    "BoatBasin": (4, 8),
    "SeaView": (5, 7),
    "IbrahimHyderi": (6, 6),
    "Gulshan": (7, 6),
}

```

```

connections = [
    ("CliftonBeach", "MazarQuaid", {"distance": 5.5, "duration": 15, "traffic": 6}),
    ("MazarQuaid", "DolmenMall", {"distance": 4.0, "duration": 10, "traffic": 5}),
    ("DolmenMall", "EmpressMarket", {"distance": 3.5, "duration": 8, "traffic": 4}),
    ("EmpressMarket", "PortGrand", {"distance": 6.0, "duration": 20, "traffic": 7}),
    ("PortGrand", "FrereHall", {"distance": 4.2, "duration": 12, "traffic": 5}),
    ("FrereHall", "KarachiZoo", {"distance": 7.5, "duration": 25, "traffic": 8}),
    ("KarachiZoo", "HawksBay", {"distance": 12.0, "duration": 40, "traffic": 9}),
    ("HawksBay", "TariqRoad", {"distance": 10.5, "duration": 30, "traffic": 7}),
    ("TariqRoad", "LuckyOneMall", {"distance": 8.5, "duration": 22, "traffic": 6}),
    ("CliftonBeach", "Saddar", {"distance": 5.0, "duration": 12, "traffic": 5}),
    ("Saddar", "BoatBasin", {"distance": 3.0, "duration": 9, "traffic": 4}),
    ("BoatBasin", "SeaView", {"distance": 2.5, "duration": 7, "traffic": 3}),
    ("SeaView", "IbrahimHyderi", {"distance": 6.5, "duration": 18, "traffic": 6}),
    ("IbrahimHyderi", "LuckyOneMall", {"distance": 15.0, "duration": 35, "traffic": 7}),
    ("Saddar", "Gulshan", {"distance": 9.5, "duration": 25, "traffic": 8}),
    ("Gulshan", "TariqRoad", {"distance": 4.5, "duration": 12, "traffic": 4}),
    ("EmpressMarket", "BoatBasin", {"distance": 7.0, "duration": 20, "traffic": 6}),
    ("PortGrand", "SeaView", {"distance": 5.5, "duration": 15, "traffic": 5}),
    ("FrereHall", "IbrahimHyderi", {"distance": 8.0, "duration": 22, "traffic": 6}),
]

```

Purpose: This block defines the coordinates of various intersections (nodes) in the city and the connections (edges) between them with attributes such as distance, duration (travel time), and traffic level.

Explanation:

- **coordinates:** A dictionary that maps intersection names to their (x, y) coordinates for visualization purposes.
- **connections:** A list of tuples where each tuple represents an edge between two intersections. Each edge also carries data like distance, travel time, and traffic level.

```
city_graph.add_nodes_from(coordinates.keys())
city_graph.add_edges_from((source, target, data) for source, target, data in connections)
```

Purpose: Adding the nodes and edges to the graph.

Explanation:

- `add_nodes_from(coordinates.keys())`: This line adds all the intersections (nodes) to the graph.
- `add_edges_from()`: This function adds the connections between intersections (edges) to the graph, with each edge also carrying its attributes (e.g., distance, duration, and traffic).

```
def shortest_route_bfs(graph, start, goal, attribute="duration"):
    visited = set()
    queue = [(start, [start], 0)]
    best_path = None
    best_cost = float("inf")

    while queue:
        current, route, total_cost = queue.pop(0)
        ...
```

Purpose: Implements the Breadth-First Search (BFS) algorithm to find the shortest route based on a specified attribute (default is "duration").

Explanation:

- `visited`: A set to track visited nodes and prevent reprocessing.
- `queue`: A list that simulates a queue for BFS traversal. It stores tuples with the current node, the route so far, and the total cost to reach that node.
- `best_path`: Tracks the shortest path found.
- `best_cost`: Tracks the minimum cost (duration) found during the BFS traversal.

```
def shortest_route_bfs(graph, start, goal, attribute="duration"):
    visited = set()
    queue = [(start, [start], 0)]
    best_path = None
    best_cost = float("inf")
```

Purpose: Implements the Breadth-First Search (BFS) algorithm to find the shortest route based on a specified attribute (default is "duration").

Explanation:

- **visited:** A set to track visited nodes and prevent reprocessing.
- **queue:** A list that simulates a queue for BFS traversal. It stores tuples with the current node, the route so far, and the total cost to reach that node.
- **best_path:** Tracks the shortest path found.
- **best_cost:** Tracks the minimum cost (duration) found during the BFS traversal.

```
def shortest_route_dfs(graph, start, goal, attribute="duration"):
    visited = set()
    stack = [(start, [start], 0)]
    best_path = None
    best_cost = float("inf")
```

Purpose: Implements the Depth-First Search (DFS) algorithm to find the shortest route based on the specified attribute (default is "duration").

Explanation:

- **visited:** A set to track visited nodes during DFS traversal.
- **stack:** A list simulating a stack for DFS traversal, storing the current node, the route, and the total cost.
- **best_path** and **best_cost:** Similar to BFS, these track the shortest path and the minimum cost found during DFS.

```

plt.figure(figsize=(12, 8))
nx.draw(
    city_graph,
    pos=coordinates,
    with_labels=True,
    node_color="lightblue",
    node_size=1500,
    font_size=10,
    font_weight="bold",
    edge_color="black",
)

highlight_path(city_graph, bfs_path, coordinates)

edge_labels = {(source, target): f"{data['duration']} min" for source, target, data in connection
nx.draw_networkx_edge_labels(city_graph, pos=coordinates, edge_labels=edge_labels, font_size=9)

plt.title("City Navigation Graph with Highlighted BFS Path")
plt.show()

```

Purpose: Visualizes the city's road network using `matplotlib` and `networkx`. The optimal BFS path is highlighted in red.

Explanation:

- `nx.draw()`: This function draws the city's graph using the node coordinates. Nodes are labeled, and edges are drawn in black.
- `highlight_path()`: A custom function that highlights the edges of the BFS path in red, making it easy to see the optimal route.
- `nx.draw_networkx_edge_labels()`: Displays the edge labels (duration) on the graph.
- `plt.show()`: Displays the graph with all the details.

2.3.7. *Results:*

BFS Optimal Path: ['CliftonBeach', 'Saddar', 'Gulshan', 'TariqRoad', 'LuckyOneMall'] Duration: 71
DFS Optimal Path: ['CliftonBeach', 'Saddar', 'Gulshan', 'TariqRoad', 'LuckyOneMall'] Duration: 71

City Navigation Graph with Highlighted BFS Path

